Testing as Requirements Definition

Concrete examples that people can agree on

Gregory S. DeLozier, PhD

Who can approve this requirement?

> The Wikipedia light bulb page should mention Edison.

Behavior-driven development (BDD) starts with examples people can discuss.

What exactly would count as passing?

Start with the test we know

```
text = browser.find_element(By.TAG_NAME, "body").text
assert "Edison" in text
```

From this chapter's `code/` folder:

```
python3 -m pytest -v test_light_bulb_search.py
```

Where is the requirement in this code?

A readable executable example

```
Feature: Inventors on Wikipedia
  Scenario: The light bulb page mentions Edison
    Given I have opened the Wikipedia home page
    When I search for Light Bulb
    Then the page text contains Edison
```

Gherkin describes the example; behave runs its Python steps.

Could someone outside the development team review this?

Three different jobs

Clause	Job	Here
Given	Establish the starting state	Wikipedia is open
When	Perform an action	Search for Light Bulb
Then	Check the result	Find Edison in the text

If Given fails, what have we learned?

Two files, one requirement

```
01-direct/
  features/
    light_bulb.feature
    steps/
      wikipedia_steps.py
```

File	Contains
Feature	The example people agree on
Steps	The Python that carries it out

The wording has to match the step definitions.

Installing the runner

From the chapter's `code/` folder, in the course environment:

```
python3 -m pip install -r requirements.txt
```

> Windows: use `py` in place of `python3` outside an activated environment. Chrome must be installed for the browser examples. Selenium Manager handles the matching driver; first use may need network access.

Given: establish the starting state

```
@given("I have opened the Wikipedia home page")
def open_home_page(context):
    context.browser = webdriver.Chrome()
    context.add_cleanup(context.browser.quit)
    context.wait = WebDriverWait(context.browser, 15)
    context.browser.get(
        "https://en.wikipedia.org/wiki/Main_Page")
```

The saved example also waits for a usable search box.

When: perform and check the action

```
start = context.browser.current_url
box = context.browser.find_element(By.NAME, "search")
box.send_keys("Light Bulb", Keys.ENTER)
context.wait.until(EC.url_changes(start))
heading = context.browser.find_element(
    By.ID, "firstHeading").text
assert not heading.startswith("Search results"), heading
```

Why wait for a URL change?

Then: check the agreed result

```
@then("the page text contains Edison")
def page_text_contains_edison(context):
    text = context.browser.find_element(
        By.TAG_NAME, "body").text
    assert "Edison" in text
```

No extra navigation hidden inside the check.

Live: run the first scenario

From `code/`:

```
cd 01-direct
python3 -m behave
```

Expected: **1 scenario passed, 3 steps passed**.

Change `Edison` to `Edisonn` in the Then step's assertion.

Which step fails? What does the report tell us?

A second example exposes repetition

```
Scenario: The helicopter page mentions Sikorsky
  Given I have opened the Wikipedia home page
  When I search for Helicopter
  Then the page text contains Sikorsky
```

From `code/02-helicopter/`: `python3 -m behave`

Expected: **2 scenarios, 6 passing steps**.

What changed? What stayed the same?

Reuse the operation; vary the data

```
@when("I search for {invention}")
def search_for_invention(context, invention):
    # The same search operation, using invention.
    ...

@then("the page text contains {inventor}")
def page_text_contains_inventor(context, inventor):
    text = context.browser.find_element(
        By.TAG_NAME, "body").text
    assert inventor in text
```

Complete code: `03-reusable/features/steps/`

One scenario, several examples

```
Scenario Outline: An invention page names its inventor
  Given I have opened the Wikipedia home page
  When I search for <invention>
  Then the page text contains <inventor>

  Examples:
    | invention  | inventor |
    | Light Bulb | Edison   |
    | Helicopter | Sikorsky |
```

From `code/03-reusable/`: `python3 -m behave`

Live: add a row, not a function

invention	inventor
Telephone	Bell
Printing Press	Gutenberg
Dynamite	Nobel

From `code/04-examples-table/`: `python3 -m behave`

Expected: **10 scenarios, 30 passing steps**.

Add a new search and an expected name. Predict the result.

Keep the setup visible

```
Background:
  Given I have opened the Wikipedia home page

Scenario Outline: An invention page names its inventor
  When I search for <invention>
  Then the page text contains <inventor>
```

From `code/05-background/`: `python3 -m behave`

Background runs before every scenario, including every row.

Hooks handle test machinery

Hook	This example
before_all	Open one browser
before_scenario	Clear cookies
after_scenario	Save a screenshot on failure
after_all	Close the browser

From `code/06-hooks/`: `python3 -m behave`

What could leak from one scenario into another?

Failure leaves something to inspect

```
def after_scenario(context, scenario):
    if scenario.status == "failed":
        name = re.sub(r"\W+", "-", scenario.name)
        # The saved version also normalizes the name.
        os.makedirs("screenshots", exist_ok=True)
        context.browser.save_screenshot(
            f"screenshots/{name}.png")
```

A screenshot helps explain a failure. It does not decide correctness.

The requirement does not need a browser

> A spreadsheet cell should accept the numeric forms we support.

Input	Agreed result
`007`	7
`.5`	0.5
`-12.5`	-12.5
`1e3`	Rejected by this version

Which decisions belong to the customer?

Make the accepted forms explicit

```
Scenario Outline: A numeric cell
  When I type "<entry>" into a cell
  Then the cell holds the number <number>

  Examples:
    | entry | number |
    | 007   | 7      |
    | .5    | 0.5    |
    | -12.5 | -12.5  |
```

The table is a small, reviewable specification.

When records what happened

```
@when('I type "{entry}" into a cell')
def enter_cell(context, entry):
    context.value = None
    context.error = None
    try:
        context.value = parse(entry)
    except ValueError as error:
        context.error = error
```

Rejection can be the expected result.

Then decides whether it was right

```
@then("the cell holds the number {number}")
def check_number(context, number):
    assert context.error is None
    assert math.isclose(context.value, float(number),
                        rel_tol=1e-12, abs_tol=1e-12)

@then("the cell is rejected as not a number")
def check_rejection(context):
    assert context.error is not None
```

The complete steps include diagnostic messages.

Live: test a disputed requirement

From `code/07-spreadsheet-parser/`:

```
python3 -m behave
```

Expected: **21 scenarios, 42 passing steps**.

Move `1e3` from the rejection table to an accepted row with `1000`.

The test fails. Is the implementation wrong, or did the requirement change?

Choose the level that answers the question

Question	Mechanism
Does a numeric string parse correctly?	Call the function
Does a Wikipedia search reach useful content?	Drive the browser
Do we agree on the required behavior?	Review concrete examples together

Use the smallest test that answers the real question.

Readable tests still need judgment

> “Edison” appears in a footnote.

The scenario passes.

Did we test what we meant?

Agree on an example. Connect it to a check. Revisit the agreement when evidence changes.